Web AI Agent Rendering Containers

Dynamic Architectures and the Need for Standardization



x

Contents

/01 The Core Trend

AI is Shifting from Generating Content to Generating UIs

/02 The Generative UI Challenges

Evaluating the Three Routes: Markdown, DSL, and Raw Web

/03 The Proposal

A Web AI Agent Subset Standard

/04 Conclusions

/01

The Core Trend

AI is Shifting from Generating Content to Generating UIs

/01 The Core Trend

From Chat to Just-in-Time UI

The paradigm is moving beyond dialog boxes to AI-orchestrated and "just-in-time" generated UI.

From Predefined Paths to Autonomy

Agents are moving from following hard-coded workflows to autonomously planning, decomposing tasks, and using tools.

From Consumer to Collaborator

The user's relationship with AI is shifting from a simple "consumer" to a "collaborator" in a Human-in-the-Loop model.

In this new paradigm, the Web serves as the critical rendering container for all visual output, from static text to fully dynamic, interactive interfaces.

The Evolution of AI Rendering: From Text to GenUI

Stage 1:
Streaming Text & Markdown

(The Initial Standard)

+ Clearer syntax for LLMs
+ Economical (fewer tokens)
+ Good separation of concerns

Stage 2:
The Workflow Paradigm (Custom Cards)

(The Initial Standard)

+ Maps user intent to pre-developed UI "cards "
+ Provides a reliable, controlled user experience

+ Brittle: Has low autonomy and is highly fragile.
+ Poor Composability: Cannot combine tasks (e.g., "order coffee" + "play music").

Stage 3:
Agentic AI & "Generative UI" (GenUI)

(The Initial Standard)

+ Breaks the rigid "intent-to-card" binding
+ AI autonomously assembles the UI
+ Uses a library of standardized, reusable components

Stage 1: Streaming Text & Markdown

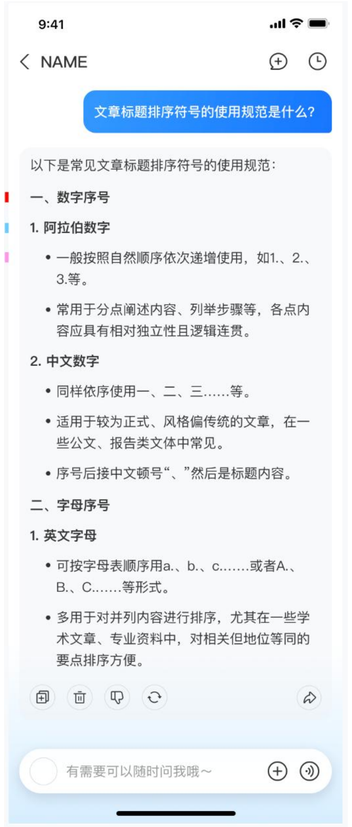
Almost all LLMs choose to support the Markdown format as their rich text output solution;

Compared to the complex nested structure of HTML, Markdown's syntax is clearer and has less ambiguity.

In the early stages when LLMs' capabilities were lower, some models might not have been able to reliably generate HTML, but they could all basically generate Markdown correctly.

Economically: Markdown brings less token consumption. Calculated using the README of the open-source project ant-design, HTML requires 5 ~ 6 times the tokens to achieve a similar rendering effect;

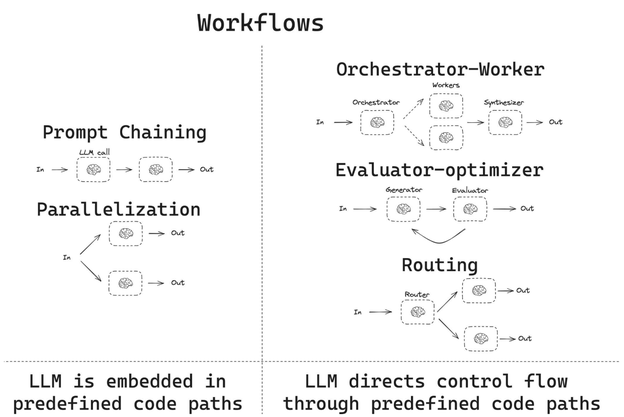
Markdown is actually a form of "separation of concerns". When LLMs output content, they only need to consider logic like "bolding" or "quoting", without needing to consider what font size is appropriate or how the quote's style should be written.



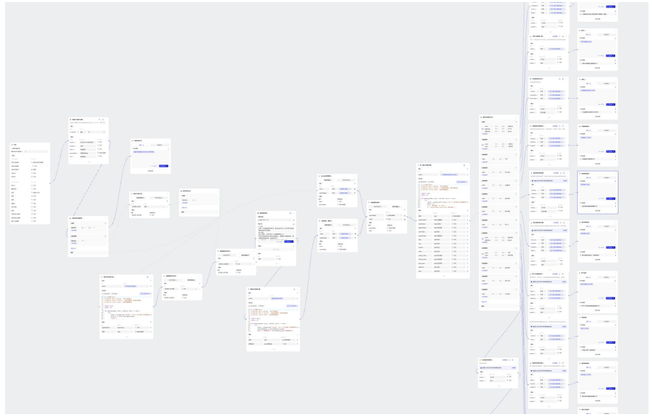


Basic markdown rendering

Stage 2: Custom Cards under the Workflow Paradigm



AI Agent Using the "Workflow" paradigm to build AI Agent



A huge commercial Workflow



Connecting branches and preset cards in the Workflow via "Card Messages"

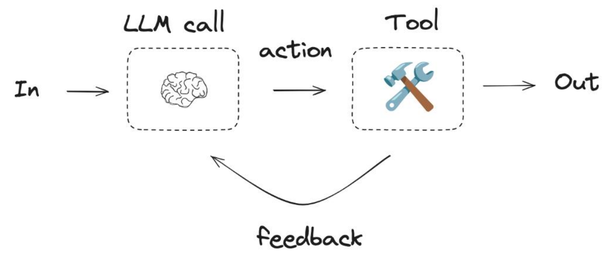
Early AI applications quickly evolved to the "Workflow" paradigm:

1. Modular Splitting: Pre-develop a series of business cards or card templates according to requirements; the main conversation container should be separated from the business cards. Cards need to define standard interfaces;

2. Dynamic Loading: Need a card discovery mechanism and dynamic loading mechanism. Code Splitting is not enough, umd or importmap is needed;

3. Communication Mechanism: Cards are not purely static; they will have their own business logic and state. Need to design a message and state synchronization plan between the card and the main conversation;

Stage 3: Agentic AI and the "Generative UI" Paradigm



LLM directs its own actions based on environmental feedback

Agentic AI emphasizes autonomy and agency , where an AI Agent can complete tasks with minimal human intervention.

The Problem: This paradigm breaks the fixed 'intent–to–card' binding. The rendering layer is forced to downgrade to simple Markdown output.

The Solution — Generative UI (GenUI):

It abandons the rigid "intent–to–card" model.

Instead, it breaks the UI down into a library of standardized, reusable components.

The Agent's task is no longer to pick a 'set meal', but to autonomously 'build' the right interface for the task, right when it's needed.

/02

The Generative UI Challenges

Evaluating the Three Technical Routes

/02 The Generative UI Challenges

The Three Routes of Generative UI

Enhanced Markdown

Pros: Simple, Streaming–friendly, Good "Separation of Concerns" .

Cons: Lacks interactivity. Fundamentally for content, not applications.

Proprietary DSLs

Pros: Enables rich, controlled "Cards" and workflows .

Cons: Ecosystem Fragmentation. Not truly "generative"; still rigid.

Raw Web (HTML/JS/CSS)

Pros: Maximum flexibility; "Infinite Canvas".

Cons: A Fallacy. This approach compounds, rather than solves, the core challenges.

/02 The Generative UI Challenges

The "Raw Web" Fallacy: Compounded Challenges

Unstructured Streaming

LLM output is token–by–token；

The container must handle incomplete tags, mismatched structures, and partial content "live" without breaking the entire UI.

Sandbox

A perfect, truly isolated "sandbox”

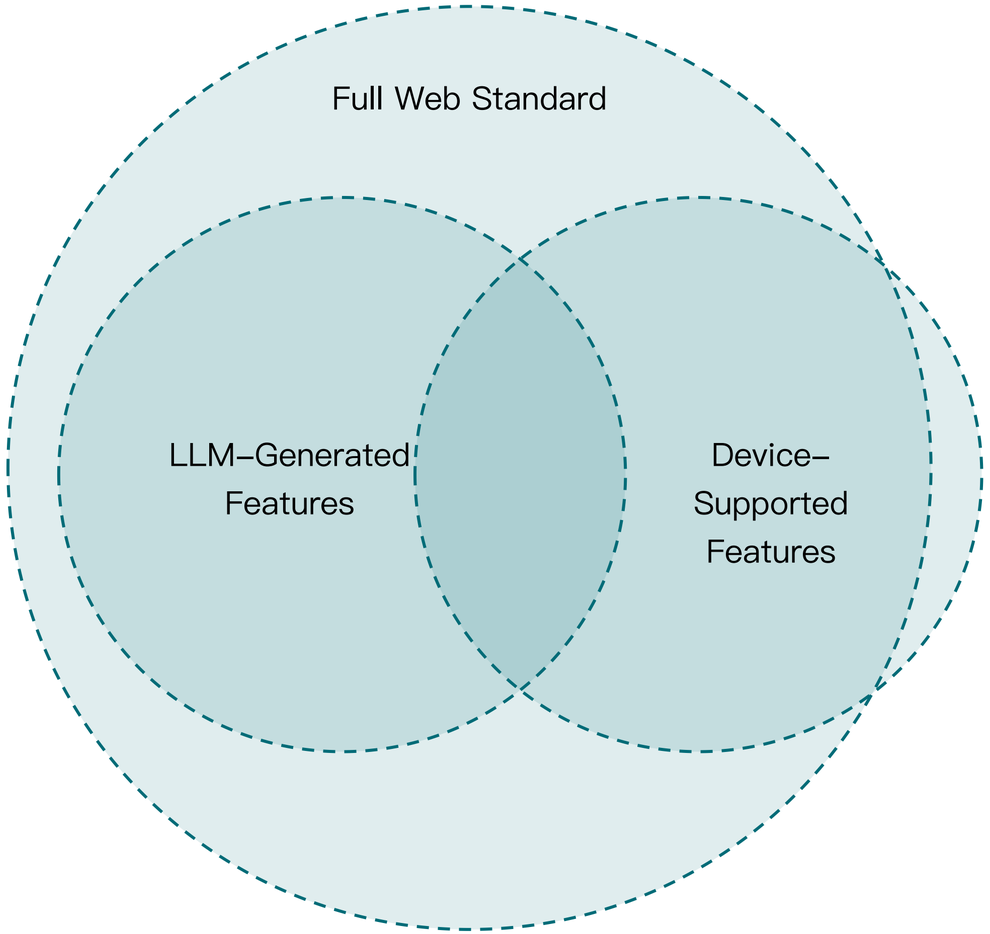
LLM Generated code is an unacceptable security and stability risk.

Modularity and Dynamics

For any complex UI, the AI would still need to compose components.

A Conversational Micro–Frontend architecture

The “Raw Web” Fallacy: Infinite Canvas vs. Limited Frame



- The "Infinite Canvas": The Web standard is a vast, evolving "infinite canvas" containing decades of specifications, features, and deprecated APIs.
- The "Limited Frame": Any user's device is a "limited frame"—a specific browser version, on a specific OS, with specific network constraints and capabilities.
- Asking an AI to "generate Raw Web" means asking it to navigate this infinite, undefined canvas. This doesn't solve the core technical challenges; it *compounds* them.

/02 The Generative UI Challenges

A "Trilemma": We Need a Fourth Way

Enhanced Markdown

Enhanced Markdown is too simple for apps.

Proprietary DSLs

Proprietary DSLs are too fragmented for an ecosystem.

Raw Web (HTML/JS/CSS)

Raw Web is too complex and insecure.

/03

The Proposal

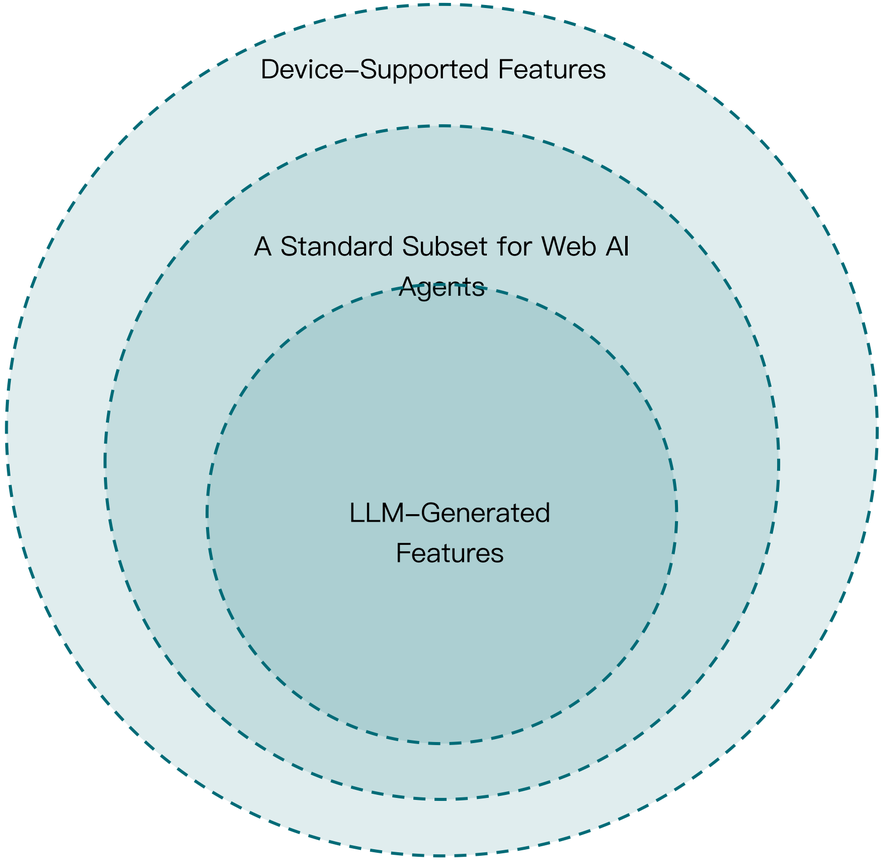
A Web AI Agent Subset Standard

/03 The Proposal

RFC: Should a subset standard for Web AI Agent be established:

- DSL is a temporary solution to solve the friction. Due to the lack of standards, various organizations reinventing DSLs leads to ecosystem fragmentation.
- The hope is to define a lean, cross-platform Web AI Agent standard;
- In the future, LLMs will no longer generate Raw Web, nor custom DSLs; but will directly generate the subset product;

The Solution: A Standard Subset for Web AI Agents

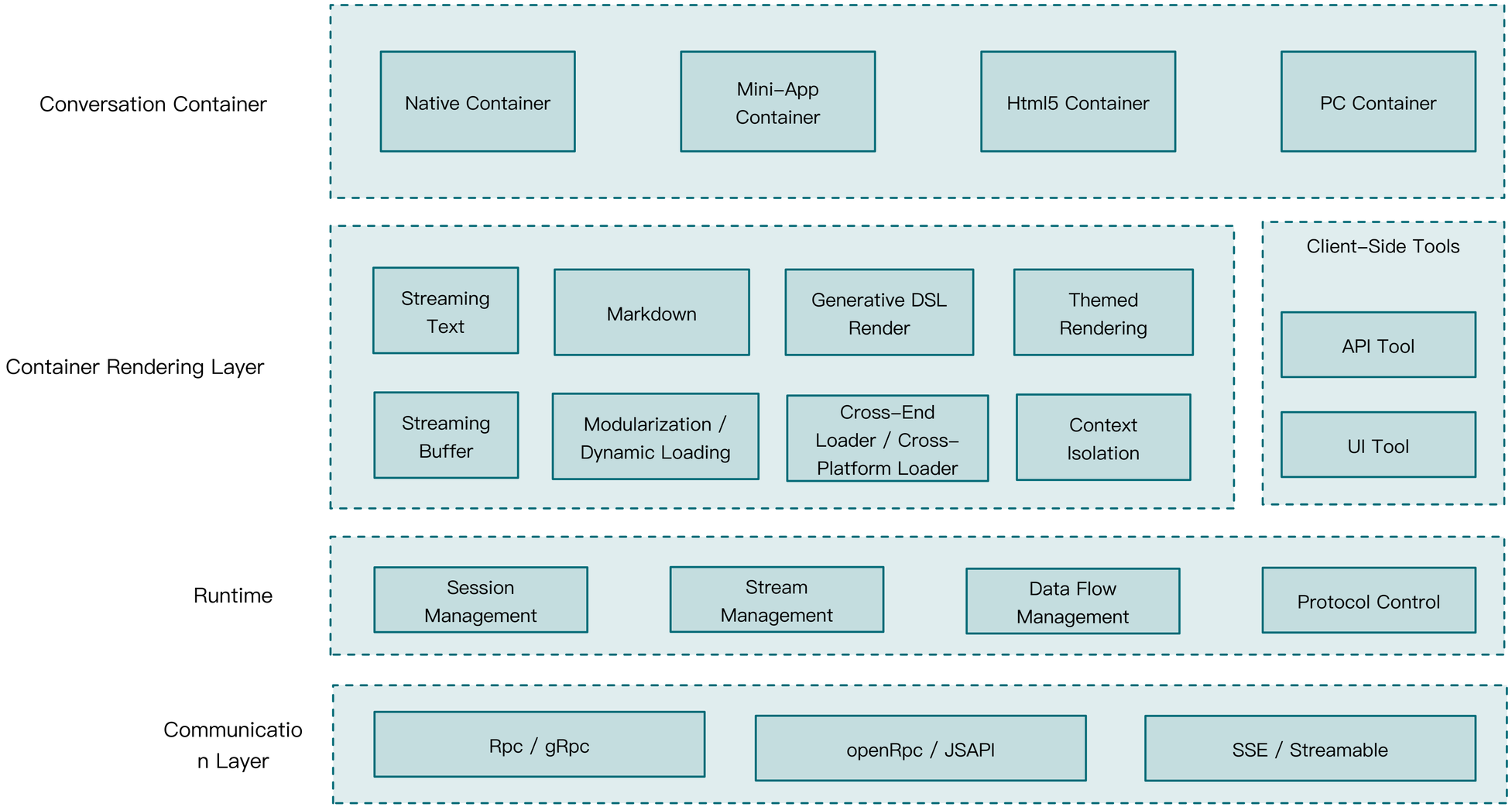


/04

Conclusions

Build Web AI Rendering Containers

/04 Conclusions



/04 Conclusions

Conclusions

"Generative AI" paradigm is expanding from text and content generation to interface and application generation: Unlike AI Coding, it is a product dynamically generated and assembled at runtime by an AI Agent based on requirements, context, and user input.

The "Generative AI" paradigm shift brings systemic friction to the Web: Current Web architecture has systemic friction with the generative paradigm demands of AI Agents in terms of performance, reliability, and security. This requires us to rethink Web standards and architectural best practices.

Mitigate friction by building AI Agent Web Containers: Web developers should proactively understand and embrace this paradigm shift to deliver a more intelligent experience to users.

Standardization organizations should establish AI Agent rendering standards: Create a set of open, extensible, secure, and platform-agnostic intermediate standards for AI Agents to generate dynamic, interactive user interfaces. This will empower developers to build interoperable, cross-platform next-generation intelligent applications and provide end-users with a consistent, reliable, and rich experience.

Thanks

/ Appendix: Intellectual Property Notice

一、 字体 / Font Copyright

本演示文档的制作使用了“OPPO Sans 4.0”字体。
This presentation document was created using the "OPPO Sans 4.0" font.

本使用行为遵循《OPPO Sans 字体许可协议》的条款，特此鸣谢。
This use complies with the terms of the "OPPO Sans Font License Agreement", and special thanks are given.

二、 图片版权 / Image Copyright

文档内所引用的所有截图、图像均来源于互联网公共资源，其著作权归原作者所有。
All screenshots and images cited in this document are from public internet resources, and their copyrights belong to the original authors.

本作品对相关素材的引用仅限于个人学习、研究或教学演示之目的，属于《著作权法》规定的合理使用范畴。
The citation of related materials in this work is limited to personal study, research, or teaching demonstration purposes, which falls within the scope of "fair use" as stipulated by the Copyright Law.

若任何内容涉及侵权，请及时联系我们予以删除。
If any content involves infringement, please contact us promptly for removal.

© 2025 All Rights Reserved.